



EVOLVING SECRET SHARING: DISTRIBUTING SECRETS IN A CHANGING WORLD

Gianfranco Vitiello, University of Salerno

Mentor: Roberto De Prisco, University of Salerno

August 2026

Abstract

A secret sharing scheme is a cryptographic primitive used to share a secret among multiple hosts in such a way that it is possible to obtain the original secret only if certain subsets of them join forces; otherwise, any attempt will be useless. An evolving secret sharing scheme is a secret sharing scheme allowing new hosts to join after the initial distribution of the secret. In this paper an overview of basic properties of secret sharing schemes is initially provided, along with how to measure their efficiency and how they can be classified according to their security guarantees. Then, evolving schemes are introduced, with a brief description of known efficiency bounds for them. Next, the most efficient implementation for $(3, \infty)$ -threshold schemes will be analysed in order to understand how it works. Finally, some real-world examples using this primitive will be shown.

1 Introduction

Secret sharing schemes were first introduced in the scientific literature in 1979, when Shamir[8] and Blakley[1] independently proposed two cryptographic schemes allowing a secret to be shared among different subjects, without the possibility to reconstruct it until a quantitative constraint is satisfied. The need of such a new cryptographic primitive is well explained in [1]. Typically, a cryptosystem relies on an important key (it could be RSA's private exponent, DES' private key and so on): if there are too many copies of it, an attacker could obtain it; on the other hand, if copies are too few (or none), there is a concrete possibility to lose the key permanently. In each case, the likelihood of an incident is very high. Incidents are classified in two main categories:

- *abnegation incident*: the guard makes the copy unusable;
- *betrayal incident*: the copy becomes known to an adversary;
- *combination incident*: a combination of the previous two types of incidents.

In order to prevent a single betrayal incident from compromising the entire security of the organization, the secret is built in such a way that each guard holds a piece of it, called *share*, and it becomes possible to reconstruct the original secret only if a certain number of them collaborate.

It is up to the company to establish how many abnegation incidents a , betrayal incidents b and combination incidents c it is willing to tolerate. In the worst case scenario (when there are no combination incidents) $a + b$ distinct incidents occur. For this reason, in order to prevent secret's theft/loss, the total number of parts in which it has to be divided, and consequentially the number of different guards holding each of them, has to be $g = a + b + 1$; on the other hand, the number of needed guards to reconstruct the secret has to be $b + 1$. In this way:

1. if a abnegation incidents occur, a parts of the key are lost forever, but the remaining $b + 1$ allow the reconstruction;
2. if b betrayal incidents occur, b parts of the key become known, but this is not enough to reconstruct the key.

This construction represents exactly what a threshold-based secret sharing scheme performs and Blakley's implementation considers the secret as a point in a $b + 1$ -dimensional space; each guard receives a hyperplane passing through that point, so that only the intersection of $b + 1$ such hyperplanes uniquely identifies it. An alternative implementation of a threshold scheme was provided by Shamir in the same year [8], this time by exploiting Lagrange interpolation. More details of both constructions will be provided in the next section, since some formal background is required beforehand.

2 Preliminary definitions and examples

The schemes discussed in the previous section were introduced simply by stating that in order to reconstruct the secret, the collaboration of at least k participants out of n is required. But what if we want to characterize in a more general way a subset of participants whose shares allow/do not allow the reconstruction the secret?

Definition 1. A subset of participants $A \subseteq \mathcal{P}_n$ is called *qualified* if it is able to reconstruct the secret.

Definition 2. A subset of participants $A \subseteq \mathcal{P}_n$ is called *prohibited* if it is unable to reconstruct the secret.

Definition 3. The set of all possible *qualified subsets* is called *access structure*.

One of the most relevant properties of an access structure is **monotonicity**, which could be defined as follows.

Definition 4. Let P be the set of participants, A and A' a subset of it, and $\mathcal{A}$ the access structure defined on it. $\mathcal{A}$ is a monotone access structure if for each A and A' such that $A \in \mathcal{A}$ and $A \subset A'$, it holds that $A' \in \mathcal{A}$.

In this paper, only monotone access structures will be considered. Anyway, with the provided definitions, it is now possible to explain what a secret sharing scheme is according to [3].

Definition 5. Let D be the domain of secrets, S be the domain of shares and n the number of participants. A secret sharing scheme Π is a pair of algorithms $(Share, Recon)$, where:

- *Share*: on input $s \in D$, outputs n shares $sh_1, \dots, sh_n \in S$;
- *Recon*: on input $\{sh_i\}_{i \in A}$ (with $A \subseteq \mathcal{P}_n$), outputs a secret $s' \in D$.

Moreover, Π must satisfy the following two properties:

- **correctness**: for each qualified set A , $Recon(\{sh_i\}_{i \in A})$ always outputs the original secret s ;
- **perfect secrecy**: for each prohibited set B , given two randomly chosen secrets $s_1, s_2 \in D$, it holds:

$$Pr[Recon(\{sh_i\}_{i \in B}) = s_1] = Pr[Recon(\{sh_i\}_{i \in B}) = s_2]$$

Anyway, since the requirement of perfect secrecy can be very strict, it is possible to relax it by requiring *computational* security: for each prohibited set B , given two randomly chosen secrets $s_1, s_2 \in D$ and a negligible function $negl(n)$, it holds:

$$\left| Pr[Recon(\{sh_i\}_{i \in B}) = s_1] - Pr[Recon(\{sh_i\}_{i \in B}) = s_2] \right| \leq negl(n)$$

An example of a simple secret sharing scheme is the following: let s be the secret to share and n the number of participants. The participant p_i will receive from the dealer a random value $r_i \in \mathbb{Z}$ for $i = 1, \dots, n-1$. The participant p_n will instead receive

$$r_n = s - \sum_{i=1}^{n-1} r_i$$

In order to reconstruct the original secret, it is necessary that all the participants collaborate. For that reason, this additive scheme is just a specific case of a threshold scheme where the threshold is equal to the number of participants.

Let n be the number of participants, k the threshold and s the secret. s is an element of the finite field $\mathbb{Z}_p$ (p is prime and $s < p$).

In the scheme provided by Blakley in [1], given the point $(x_1, \dots, x_k) \in \mathbb{Z}_p^k$, the secret s corresponds to the coordinate x_1 while the other ones are chosen randomly. For each participant $i = 1, \dots, n$, the procedure for generating the associated share is the following:

1. a linear equation in k unknowns $y_1, \dots, y_k$ passing through the point $(x_1, \dots, x_k)$ is generated:

$$a_{i,1}y_1 + a_{i,2}y_2 + \dots + a_{i,k}y_k = b_i$$

the coefficients $a_{i,1}, \dots, a_{i,k}$ are chosen randomly, subject to the constraint that the point $(x_1, \dots, x_k)$ satisfies the equation;

2. b_i is calculated by substitution of the unknowns with the coordinates of the point initially generated;
3. the share of the participant i is $(a_{i,1}, \dots, a_{i,k}, b_i)$.

The reconstruction process consists in solving a system of k equations in k variables, which has only one possible solution. If only $k - 1$ participants try to reconstruct the secret, the resulting system is underdetermined, and every one of the p possible values of the secret is equally likely.

As for the construction provided by Shamir in [8], the share generation process works as follows:

1. a polynomial $f(x)$ of degree $k - 1$ with coefficients in $\mathbb{Z}_p$ is chosen in such a way that $f(0) = s$:

$$s + a_1x^1 + a_2x^2 + \dots + a_{k-1}x^{k-1}$$

where $a_1, \dots, a_{k-1}$ are random values in $\mathbb{Z}_p$;

2. the participant i receives the share $sh_i = f(i)$.

The reconstruction process exploits Lagrange interpolating polynomial: given at least k points (i, sh_i) the following formula is used:

$$s = \sum_{i=1}^k sh_i \cdot \prod_{j=1, j \neq i}^k \frac{-j}{i-j} \pmod{p}$$

Regarding the share size, we can notice that it is equal to the secret size, so $\rho = 1$ which means that Shamir's scheme is optimal.

3 Access structure types

All the proposed schemes in the previous section are threshold-based. One may wonder if it is possible to build a secret sharing scheme with an access structure that relies on non-quantitative constraints. The answer to this question is yes, as shown in [4], where it is proved that it is possible to define a secret sharing scheme for every monotone access structure. So the problem now becomes how to build such schemes efficiently. But how do we measure the efficiency of a secret sharing scheme? One of the possible metrics is the *share size*.

Definition 6. The share size of a secret sharing scheme is equal to the number of bits of the longest distributed share.

Two other metrics used quite often in literature are the *information rate* ρ and its reciprocal. The information rate is defined as the secret size divided by the longest share size and an ideal scheme has an information rate equal to 1. One of the most important resources to have an idea of how difficult it is to build efficient schemes for certain access structures is [11]. In that paper, D.R. Stinson considers a set of four participants $\mathcal{P} = \{P_1, P_2, P_3, P_4\}$ and for each possible access structure shows how to build a secret sharing scheme in the most efficient way. In fact, he shows that there are certain access structures for which an ideal scheme cannot exist.

One of the multiple ways to classify secret sharing schemes regards the security guarantees they offer. In particular, S. Kreen et al.[7] distinguish four main categories:

- **basic secret sharing**, where the assumption is that both the dealer and participants are honest. They can be divided into the following subcategories:
 - perfectly private secret sharing, which relies on the perfect secrecy assumption;
 - computationally private secret sharing, in which the perfect secrecy requirement is relaxed in order to obtain more efficient schemes;
 - ramp secret sharing, a trade-off between small share sizes and unconditional security. In ramp schemes, sets of shares are not only qualified or non-qualified; there are also intermediate sets which cannot recover the secret, but may leak partial information about it;
- **robust secret sharing**, which remains secure even in the presence of malicious participants trying to prevent the reconstruction of the secret. Concretely, a robust secret sharing scheme guarantees that given t shares, of which at most c may have been maliciously modified by an adversary, reconstruction will still result in the original secret s . In that way, not only is confidentiality granted, but also the integrity of the data;
- **verifiable secret sharing**, providing certain guarantees even if the dealer is corrupted. Precisely, such schemes allow the parties to jointly check the consistency of the shares they received, such that a user is guaranteed to receive the same secret from any sufficiently large subset of shares;
- **proactive secret sharing**, which maintains long-lasting confidentiality by renewing the shares from time to time.

4 Evolving secret sharing

The schemes analysed until now were considering static access structures, i.e. predetermined before the generation of the shares. If another participant joins the company after the distribution of the shares, the shares have to be recomputed. In order to avoid this kind of situation, there exist secret sharing schemes for evolving access structures.

Definition 7. An **evolving** access structure $\mathcal{A}$ is a set of subsets of $\mathcal{P}_n$ which only grows in

cardinality over time.

Evolving secret sharing was first introduced in a 2016 work by Komargodski et al. and then extended in 2018 [5]. Even in an evolving environment, it is possible to build threshold-based schemes. The minimum threshold is obviously 2 and the technique adopted by Komargodski et al. for sharing a 1 bit secret uses prefix codes. A prefix code is a function:

$$\Sigma : \mathbb{N} \rightarrow \{0, 1\}^*$$

such that for any two inputs $t_1, t_2 \in \mathbb{N}$ such that $t_1 \neq t_2$, $\Sigma(t_1)$ is not the prefix of $\Sigma(t_2)$ and vice versa. Let $\sigma(t)$ be the length of the codeword $\Sigma(t)$, $s \in \{0, 1\}$ the secret to be shared and w an infinite random binary string. The scheme works as follows:

- *Share*: on input w and n , the dealer outputs the shares $sh_1, \dots, sh_n$. In particular, the t^{th} share sh_t is equal to:
 1. $w_{[\sigma(t)]}$ (the first $\sigma(t)$ bits of w), if $s = 0$;
 2. $\Sigma(t) \oplus w_{[\sigma(t)]}$, if $s = 1$.
- *Recon*: on input sh_{t_1} and sh_{t_2} , if sh_{t_1} is a prefix of sh_{t_2} , outputs 0; otherwise, outputs 1.

Since we don't know how many participants take part in the scheme, the definition of share size provided in the previous section cannot be applied for evolving schemes.

Definition 8. The share size of an evolving secret sharing scheme is the length in bits of the share of the next participant.

So, with this definition of share size, if we use Elias's code prefix, it turns out that $\sigma(t) = \log_2 t + 2 \log_2 \log_2 t + 2$.

What if we want to share a secret longer than 1 bit? The idea is to combine a secret sharing scheme with static access structure and an evolving secret sharing scheme, even with poor efficiency. More precisely, participants are divided into generations of size 2^g so that participant t belongs to generation number $\lfloor \log_2 t \rfloor$. Among parties of the same generation we produce the shares by using Shamir's scheme, whose share size is roughly $\log_2 t$ bits. When a generation begins the dealer prepares shares for all its members. Moreover, a 2-threshold scheme is used to generate one share per generation that will be distributed among all its members. More precisely:

1. the secret s is split using a $(2, \text{size}(g))$ -threshold scheme; the resulting shares are called $u_1^{(g)}, \dots, u_{\text{size}(g)}^{(g)}$;
2. an additional share $v^{(g)}$ is generated starting from the secret and previous "generational" shares $\{v^{(i)}\}_{i \in \{0, \dots, g-1\}}$ produced by the evolving 2-threshold scheme;
3. the share associated to the j^{th} participant in the g^{th} generation is

$$(u_j^{(g)}, v^{(g)})$$

In that way, given two participants t_1 and t_2 , reconstruction is always possible:

- if they belong to the same generation g , the shares coming from the Shamir's scheme are used;
- if they belong to different generations g_1 and g_2 , the shares coming from the 2-threshold scheme are used.

This reasoning generalizes to arbitrary threshold k : the g -th generation has $2^{(k-1)(g+1)} - 2^{(k-1)g} \leq t \cdot 2^{k-1}$ parties and participant t_i belongs to generation $g = \lfloor \frac{\log t_i}{k-1} \rfloor$.

When generation g begins, the dealer:

1. splits s using a $(k, \text{SIZE}(g))$ -threshold scheme, producing shares $u_1^{(g)}, \dots, u_{\text{SIZE}(g)}^{(g)}$;
2. generates $k-1$ shares $v_1^{(g)}, \dots, v_{k-1}^{(g)}$ of the evolving k -threshold scheme Π ;
3. for each $i \in [k-1]$, splits $v_i^{(g)}$ using a $(i, \text{SIZE}(g))$ -threshold scheme, producing shares $w_{i,1}^{(g)}, \dots, w_{i,\text{SIZE}(g)}^{(g)}$;
4. the j -th party of generation g receives $(u_j^{(g)}, w_{1,j}^{(g)}, \dots, w_{k-1,j}^{(g)})$.

As for the reconstruction process, if all k parties come from the same generation, they use their $u^{(g)}$ shares to recover s . Otherwise, letting c_i be the number of parties from generation i (with $\sum_i c_i = k$, each $c_i \leq k-1$), any c parties from generation g recover $v_c^{(g)}$ via their $w_{c,\cdot}^{(g)}$ shares. Collecting one contribution per generation yields k shares of Π (since $\sum_i c_i = k$), from which s is reconstructed.

Moreover, in the same work, Komargodski et al. gave an upper bound on the share size of an evolving k -threshold scheme:

Theorem 1. For every $k, l \in \mathbb{N}$, there is a secret sharing scheme for the evolving k -threshold access structure and an l -bit secret in which for every $t \in \mathbb{N}$ the share size of the t^{th} party is

$$(k-1) \log_2 t + \text{poly}(k, l) \cdot o(\log_2 t)$$

In the case of $k=2$, they also provided a lower bound coming from the following reasoning: there is a one-to-one correspondence between prefix codes and evolving 2-threshold secret sharing schemes. Since there exists a lower bound for the length of prefix codes, this also applies to the share size.

Corollary 1. For any constants $c, l \in \mathbb{N}$, there is no secret sharing scheme for the evolving 2-threshold access structure in which for every $t \in \mathbb{N}$ the share size of the t^{th} party is at most

$$\log_2 t + \log_2 \log_2 t + \dots + \log_2^{(l)} t + c = \log_2 t + o(\log_2 t)$$

where $\log_2^{(i)}$ is the i -times repeated $\log_2$ of t .

If we compare these bounds with the share size of the prefix-codes-based scheme, it turns out that it is almost optimal and trying to improve it is like looking for a needle in a haystack.

5 $(3, \infty)$ –secret sharing

As we saw in the previous section, it is useless and highly demanding to try to find a better lower bound for $(2, \infty)$ schemes. But what about $(3, \infty)$ schemes? Before showing what is the state of art in that case, let me introduce a simple $(3, \infty)$ –threshold secret sharing scheme: let s be the secret and $r_i^{\{i\}}$ and $r_i^{\{i,k\}}$ for $i = 1, 2, \dots$ and $k = 1, \dots, i - 1$ random values. When participants take part in the scheme, they obtain a share according to the following distribution rule:

- $p_1 \leftarrow r_1^{\{1\}}$
- $p_2 \leftarrow r_2^{\{2\}}, r_2^{\{1,2\}}$
- $p_3 \leftarrow r_1^{\{1\}} \oplus r_2^{\{1,2\}} \oplus s, r_3^{\{3\}}, r_3^{\{1,3\}}, r_3^{\{2,3\}}$
- $p_4 \leftarrow r_1^{\{1\}} \oplus r_2^{\{1,2\}} \oplus s, r_1^{\{1\}} \oplus r_3^{\{1,3\}} \oplus s, r_2^{\{2\}} \oplus r_3^{\{2,3\}} \oplus s, r_4^{\{4\}}, r_4^{\{1,4\}}, r_4^{\{2,4\}}, r_4^{\{3,4\}}$
- ...
- $p_i \leftarrow \{r_j^{\{j\}} \oplus r_k^{\{j,k\}} \oplus s\}$ for $j < k < i$, $r_i^{\{i\}}$ and $r_i^{\{i,j\}}$ for $j < i$.

If we consider the share size of the next participant, we can see that it grows quadratically with regard to the bit length of the secret. This is of course not efficient at all. A major improvement comes from using the Komargodski et al.’s general construction, which decreases the share size to:

$$2 \log_2 t + \text{poly}(3, l) \cdot o(\log_2 t)$$

But can we do better? Yes, in 2021, D’Arco et al.[3] provided an implementation relying on two building blocks:

- the previously shown $(3, \infty)$ –threshold scheme with quadratic complexity, that will be called $\mathcal{S}^\infty$;
- a threshold-based secret sharing scheme with a fixed number of participants, that will be called $\mathcal{S}^i$.

As we can see, the trick is the same used by Komargodski et al.: each participant is assigned to a generation with the difference that now generation i consists of 2^{4^i} participants. Let $n_0 = 0$ and $n_i = 2^{4^i}$: the generation g^i starts from party $n_{i-1} + 1$ and ends with party n_i . So, let $\bar{n}_i = n_i - n_{i-1}$ be the cardinality of the i –th generation and $N^i = \bar{n}_i + 1$ (for $i \geq 1$). It turns out that generation 1 has 16 members, while generation 2 already has 65536 members and this greatly improves the scalability of the algorithm.

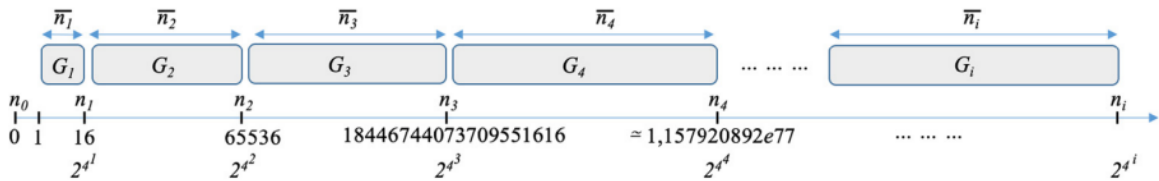


Figure 1: Generations’ sizes

One possible way to implement the scheme $\mathcal{S}^i$ (the one used for the members of the i –th generation) is by using the Asmuth-Bloom scheme, which is one of the most efficient solutions in the literature.

Let k be the threshold and $p_0, p_1, \dots, p_{N^i}$ public primes such that $p_0 \cdot p_{N^i-(k+2)} \cdot p_{N^i-(k+3)} \cdot \dots \cdot p_{N^i} < p_1 \cdot \dots \cdot p_k$ and let I be the interval $[0, p_1 \cdot \dots \cdot p_k)^1$. The construction is the following:

- *Share*: on input $s \in \mathbb{Z}_{p_0}$, choose randomly a value α such that $s + \alpha \cdot p_0 < p_1 \cdot \dots \cdot p_k$ and calculate $s' = s + \alpha \cdot p_0 \in I$ (the secret s is shifted) and output $sh_j = s' \bmod p_j$ for $j = 1, \dots, N^i$;
- *Recon*: on input $\{sh_j\}_{j \in A}$, solve

$$\begin{cases} x \equiv sh_{j_1} \pmod{p_{j_1}} \\ \vdots \\ x \equiv sh_{j_k} \pmod{p_{j_k}} \end{cases}$$

and provide in output $s = x \bmod p_0$;

5.1 Construction

At this point, we are ready to explain the construction defined in [3]. But before we do it, two key intuitions are given to facilitate the comprehension:

1. to each generation g a random value RS^g is associated;
2. the scheme $\mathcal{S}^i$ produces one extra share sh_E which is provided to all the participants of past generations as it is, and masked with the random strings of past generations to all the participants of the current generation;²

Let $s \in S$ be the secret and g the current generation. The scheme works as follows:

- *Share*: on input s and random strings $RS^1, \dots, RS^{g-1}$, output:
 - the share sh_g^∞ by using the *Share* function of the scheme $\mathcal{S}^\infty$;
 - the shares $sh_1^g, \dots, sh_{n_g}^g, sh_E^g$ by using the *Share* function of the scheme $\mathcal{S}^g$;
 - the masked shares $RS^j \oplus sh_E^g$, for $j = 1, \dots, g-1$;
 - a random string RS^g .
- *Recon*: on input $\{sh_j\}_{j \in A}$ (with $A \subseteq \mathcal{P}_n$), output a secret $s' \in S$ according to the correctness and privacy properties.

Since it could be a little bit confusing, let t be the t^{th} participant, belonging to the generation $g = \lceil \log_4 \log_2 t \rceil$ and identified by the index i in its generation. The share of t is structured in five different parts:

- **P1**: sh_g^∞ ;
- **P2**: $sh_E^1, sh_E^2, \dots, sh_E^{g-1}$;
- **P3**: sh_i^g ;
- **P4**: $RS^1 \oplus sh_E^g, RS^2 \oplus sh_E^g, \dots, RS^{g-1} \oplus sh_E^g$;

¹this condition is essential in order to guarantee that the system solved to reconstruct the key has one and only solution

²in the original version of the paper, the scheme produces two extra shares sh_B and sh_F , one propagated backward and one propagated forward. Unfortunately, this construction has a security problem, as shown by Cheng et al.[2] that can be easily fixed by using one single share sh_E

- **P5:** RS^g .

Obviously, if $g = 1$, then **P2** and **P4** are absent.

5.2 Correctness

In order to prove the correctness property, we have to show that it is possible to reconstruct the original secret s in any case that involves at least three parties. Let $g(t)$ be the function that outputs the generation to which participant t belongs and $t_1, t_2, t_3 \in \mathcal{P}_n$ ordered by the moment of joining the scheme. Four scenarios are possible:

1. they all belong to the same generation, i.e. $g(t_1) = g(t_2) = g(t_3)$;
2. they all belong to different generations, i.e. $g(t_1) \neq g(t_2) \neq g(t_3)$;
3. t_1 and t_2 belong to the same earlier generation than t_3 , i.e. $g(t_1) = g(t_2) \neq g(t_3)$;
4. t_2 and t_3 belong to the same later generation than t_1 , i.e. $g(t_1) \neq g(t_2) = g(t_3)$.

Let g_1, g_2, g_3 be the generations to which respectively t_1, t_2, t_3 belongs to and i_1, i_2, i_3 the index assigned respectively to t_1, t_2, t_3 inside their generations.

5.2.1 Scenario 1

In this scenario, t_1, t_2, t_3 all belong to the same generation g . This means that to reconstruct the secret they can use the $\mathcal{S}^g$ scheme, and more precisely their **P3** part of the share.

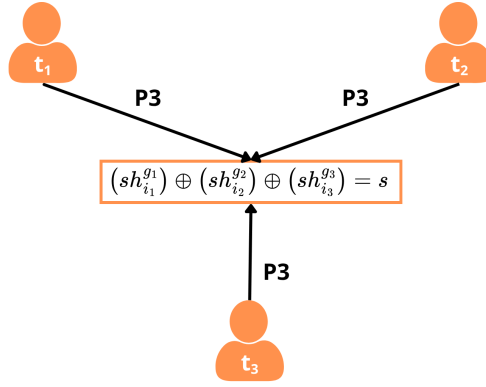


Figure 2: Reconstruction process when $g(t_1) = g(t_2) = g(t_3)$

5.2.2 Scenario 2

In this scenario, t_1, t_2, t_3 all belong to different generations. This means that to reconstruct the secret they can use the $\mathcal{S}^\infty$ scheme, and more precisely their **P1** part of the share.

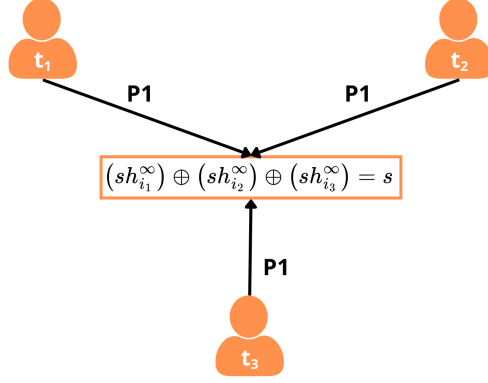


Figure 3: Reconstruction process when $g(t_1) \neq g(t_2) \neq g(t_3)$

5.2.3 Scenario 3

In this scenario, t_1 and t_2 belong to the same earlier generation than t_3 . This means that to reconstruct the secret they can use the $\mathcal{S}^{g_1}$ scheme, and more precisely t_1 and t_2 use the **P3** part of their shares while t_3 uses the extra share received for generation g_1 as part of **P2** piece of the share.

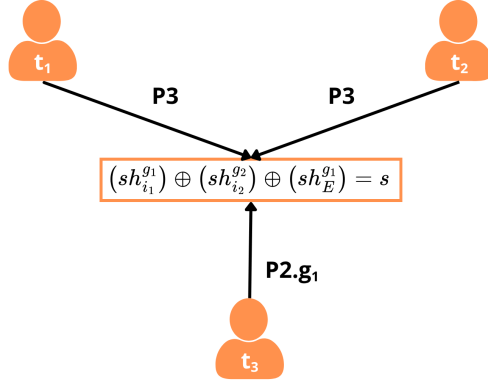


Figure 4: Reconstruction process when $g(t_1) = g(t_2) \neq g(t_3)$

5.2.4 Scenario 4

In this scenario, t_2 and t_3 belong to the same later generation than t_1 . This means that to reconstruct the secret they can use the $\mathcal{S}^{g_2}$ scheme, and more precisely t_2 and t_3 use the **P3** part of their shares, with one of them using also the extra share of its generation masked with the random string of the generation g_1 (being part of the **P4** part of the share), while t_1 uses the random string received as **P5** part of the share.

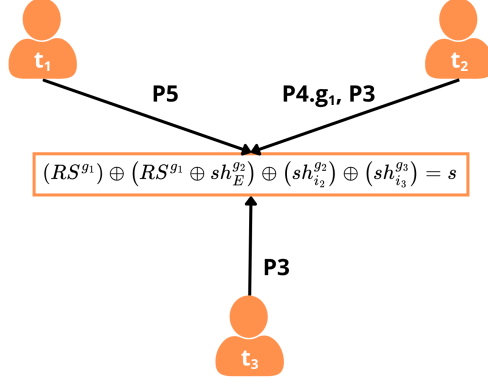


Figure 5: Reconstruction process when $g(t_1) \neq g(t_2) = g(t_3)$

5.3 Security

In order to prove the security property, it is necessary to show that in any case in which the number of participants n is less than the threshold, i.e. $n \leq 2$, it is possible neither to reconstruct the original secret nor derive any information about it. Without loss of generality, we can consider the case when $n = 2$ since it represents the maximum cardinality of a prohibited set. Let t_1 and t_2 be the parties trying to reconstruct the secret. There are two possible scenarios:

1. t_1 and t_2 belong to the same generation g ;
2. t_1 belongs to an earlier generation than t_2 .

5.3.1 Scenario 1

In this scenario, t_1 and t_2 belong to the same generation g , so the pieces **P1**, **P2**, **P4** and **P5** of the share are equal. The only difference regards the piece **P3**, which represents the share assigned by the $\mathcal{S}^g$ scheme (the intragenerational one). Since the participants are only two, they have just two shares of that scheme: even if they combine them, they will be unable to reconstruct the secret.

5.3.2 Scenario 2

In this scenario, t_1 and t_2 belong to different generations, respectively g_1 and g_2 . For that reason, their shares completely differ. They can try to obtain the secret s using the reconstruction process of both $\mathcal{S}^\infty$ and $\mathcal{S}^{g_1}$ schemes. Unfortunately, this is not possible because:

- if they try to obtain the secret by using the reconstruction function of the $\mathcal{S}^\infty$ scheme, they have to use their **P1** pieces. Since only two shares are available and at least three are required, pooling them they does not allow reconstruction of the secret;
- if they try to obtain the secret by using the reconstruction function of the $\mathcal{S}^{g_1}$: t_1 can provide the share **P3** while t_2 can provide the share **P2.g1**. Since they have only two shares and at least three are needed, if they pool them, they will be unable to reconstruct the secret;

The impossibility to obtain the original secret is due to the following reasons:

1. usage of fresh randomness;
2. security of the $\mathcal{S}^g$ schemes;

3. security of the $\mathcal{S}^\infty$ scheme;
4. total independence among the used schemes.

5.4 Share size

In order to calculate the share size of the next participant to the scheme it is necessary to add the size of the pieces it gets. Let t be the t^{th} participant and $g = \lceil \log_4 \log_2 t \rceil$ its generation. As shown before, its share consists of five parts:

- **P1.** a share of the $\mathcal{S}^\infty$ scheme;
- **P2.** an extra share sh_E^i for each previous generation $i = 1, \dots, g - 1$;
- **P3.** a share of the $\mathcal{S}^g$ scheme;
- **P4.** a string obtained by masking the extra share sh_E^g with the random string RS^i , for $i = 1, \dots, g - 1$;
- **P5.** a random string RS^g .

In order to calculate the share size, some assumptions have to be made:

1. the intergenerational scheme used is the one presented at the beginning of the section (the one with quadratic complexity);
2. the intragenerational scheme used is the one presented by Asmuth-Bloom.

What turns out in [3] is that the final share size is at most:

$$c \cdot (\log_4 \log_2 t)^2 + \frac{4}{3} \sqrt[4]{4} \log_2 t + \log_2 p \cdot (\log_4 \log_2 t)$$

with c , p and z some constant values. In the case of z , the bigger it is the more wide is the interval to select prime numbers for the Asmuth-Bloom scheme.

If we consider the theorem of Komargodski et al. for general evolving k -threshold schemes, it follows that for $k = 3$ the upper bound is:

$$2 \cdot \log_2 t + \text{poly}(3, l) \cdot o(\log_2 t)$$

Since both the expressions are dominated by the first term, all the others can be discarded and it turns out that, for $z \rightarrow +\infty$, the existing upper bound is improved.

6 Real-world applications

One of the typical examples used to justify the existence of secret sharing schemes regards the access to a bank vault or the decision of dropping an atomic bomb. Both are potentially very interesting use cases of secret sharing schemes, but there are no clues that this kinds of operations are carried out by using secret sharing schemes.

At this point, one may wonder if secret-sharing schemes are just a theoretical topic or whether they have some useful usages in real life. Of course they do, and as we will see, the entire security of the Internet and the secrecy of e-votes rely on secret sharing schemes.

6.1 DNSSEC

Every time you surf the Web, instead of typing the address of the server on which the resource you are looking for is stored, you simply write the URL at which you can find it. This is possible thanks to a protocol able to translate strings into IP addresses, the Domain Name Server (DNS). Like every protocol designed for the first versions of the Internet, DNS was not designed with security as the first concern. In fact it can suffer from MITM attacks like Kashpureff and Kaminsky attacks that try to modify DNS queries with the aim of redirecting future traffic to malicious servers [9].

In order to counter any attempt at cache poisoning, a secure version of DNS called DNSSEC has been designed. That security is achieved by adding digital signatures to DNS records. To make this possible, it is necessary to rely on a PKI (public-key infrastructure), organized as follows: each DNS zone server owns a key pair: a **Zone Signing Key (ZSK)** that signs the actual records, and a **Key Signing Key (KSK)** that signs the ZSK itself.

The KSK's public part is published as a DS record in the parent zone, creating a **chain of trust** from the root zone down to the specific domain. A resolver verifies each signature going up the chain, and if any signature fails to verify, the response is rejected. In this way, it is possible to guarantee authenticity and integrity, but not confidentiality (since there is no traffic encryption).

Since the root KSK is the anchor of trust for all of DNSSEC, it must never exist as a single, easily stolen secret. For these reasons, ICANN (which manages the DNS root) splits control of the KSK private key among multiple trusted individuals, called Recovery Key Share Holders, using a (k, n) -threshold Shamir's scheme³. This means the key can only be reconstructed during official, highly audited key ceremonies, where a quorum of share holders must physically come together with their smart cards. Secret sharing therefore plays a pivotal role, even if indirectly, in the security of the Internet infrastructure.

6.2 Multi-Party Computation

Secret sharing schemes are a foundational primitive of **Multi-Party Computation**. MPC was first introduced in 1982 as 2PC by Andrew C. Yao [12] in order to solve the so-called "billionaires' problem": two billionaires want to know who is richer without revealing the amount of their wealth. One of the most important applications of MPC across large-scale industries regards secure data sharing and analysis. Let's assume a hospital collects data regarding its patients. In order to improve the health service, the hospital collaborates with a research centre that develops prediction models. To build it, sensitive data are required but since it is located on the EU ground, those sensitive data are protected by laws like the GDPR. A solution that allows collaboration despite strict legislation is MPC: instead of transferring raw patient records, the data are first split into cryptographic shares using a secret sharing scheme, and only these shares are sent to the computation nodes. Both the research centre and the hospital evaluate the desired function (which can be training a logistic regression model or computing aggregate statistics) directly on the shares, so that the final prediction model is obtained without any single party ever reconstructing another party's original data. A concrete instance of this approach is documented in [10], where LMU University Hospital and Charité – Universitätsmedizin Berlin deployed a genuine, fully-fledged secure

³More info can be found here: <https://www.iana.org/dnssec> and here: <https://www.cloudflare.com/learning/dns/dnssec/root-signing-ceremony/>

multiparty computation pipeline for an international clinical study. The authors report this as the first documented use of genuine SMPC in such a setting. They also publish the full set of GDPR-compliant technical and administrative safeguards that allowed national and European regulators to approve the pilot, effectively providing a blueprint that other hospital-research collaborations can replicate.

7 Conclusions and further considerations

After this long but incomplete journey through secret sharing schemes, what we saw is that this cryptographic primitive was introduced to solve a practical problem related to safeguarding a sensitive secret while managing multiple copies of it. The intuition was to split the secret in multiple parts to be distributed to trusted entities in such a way that it becomes possible to reconstruct the secret only by satisfying certain constraints it becomes possible to reconstruct it. Then, different types of access structures were analysed and the most fascinating thing is that what we considered is just a drop in the ocean compared to the huge literature around this kind of schemes. For example, there is the possibility to define evolving access structures with a dynamic threshold [6], which means that the threshold can become greater during the execution of the algorithm, but with the constraint that already qualified subsets will continue to be qualified even if their cardinality is lower than the new value of the threshold. After this, evolving access structures were introduced with particular attention to evolving threshold schemes. In this field, a lot of results proved by Komargodski still hold, but the upper bound provided in [3] for $(3, \infty)$ -threshold schemes improves one of them. Finally, some concrete applications of the secret sharing primitive were illustrated like the one regarding the DNSSEC which allows us to use the Internet in a more safe way (even if indirectly).

References

- [1] G. Robert Blakley. Safeguarding cryptographic keys. *1979 International Workshop on Managing Requirements Knowledge (MARK)*, pages 313–318, 1979.
- [2] Qi Cheng, Hongru Cao, and Sian-Jheng Lin. A construction of evolving 3-threshold secret sharing scheme with perfect security and smaller share size, 2024.
- [3] Paolo D’Arco, Roberto De Prisco, and Alfredo De Santis. Secret sharing schemes for infinite sets of participants: A new design technique. *Theoretical Computer Science*, 859:149–161, 2021.
- [4] M. Ito, A. Saito, and T. Nishizeki. Secret sharing scheme realizing general access structure. *GLOBECOM’87*, pages 99–102, 1987.
- [5] I. Komargodski, M. Naor, and E. Yogev. How to share a secret, infinitely. *IEEE Transactions on Information Theory*, 64:4179–4190, 2018.
- [6] Ilan Komargodski and Anat Paskin-Cherniavsky. Evolving secret sharing: Dynamic thresholds and robustness. In Yael Kalai and Leonid Reyzin, editors, *Theory of Cryptography*, pages 379–393. Springer International Publishing, 2017.
- [7] Stephan Krenn and Thomas Lorünser. *An Introduction to Secret Sharing*. SpringerBriefs in Information Security and Cryptography. Springer, 2023.
- [8] Adi Shamir. How to share a secret. *Communications of the ACM*, 22:612–613, 1979.

- [9] Soeul Son and Vitaly Shmatikov. The hitchhiker’s guide to dns cache poisoning. In *Lecture Notes of the Institute for Computer Sciences, Social-Informatics and Telecommunications Engineering*, volume 50, pages 466–483, 2010.
- [10] Fabian Spitz et al. Privacy-friendly evaluation of patient data with secure multiparty computation in a european pilot study. *npj Digital Medicine*, 2024.
- [11] D.R. Stinson. An explication of secret sharing schemes. *Designs, Codes and Cryptography*, 2:357–390, 1992.
- [12] Andrew Chi-Chih Yao. Protocols for secure computations. In *Proceedings of the 23rd Annual Symposium on Foundations of Computer Science (FOCS)*, pages 160–164. IEEE, 1982.